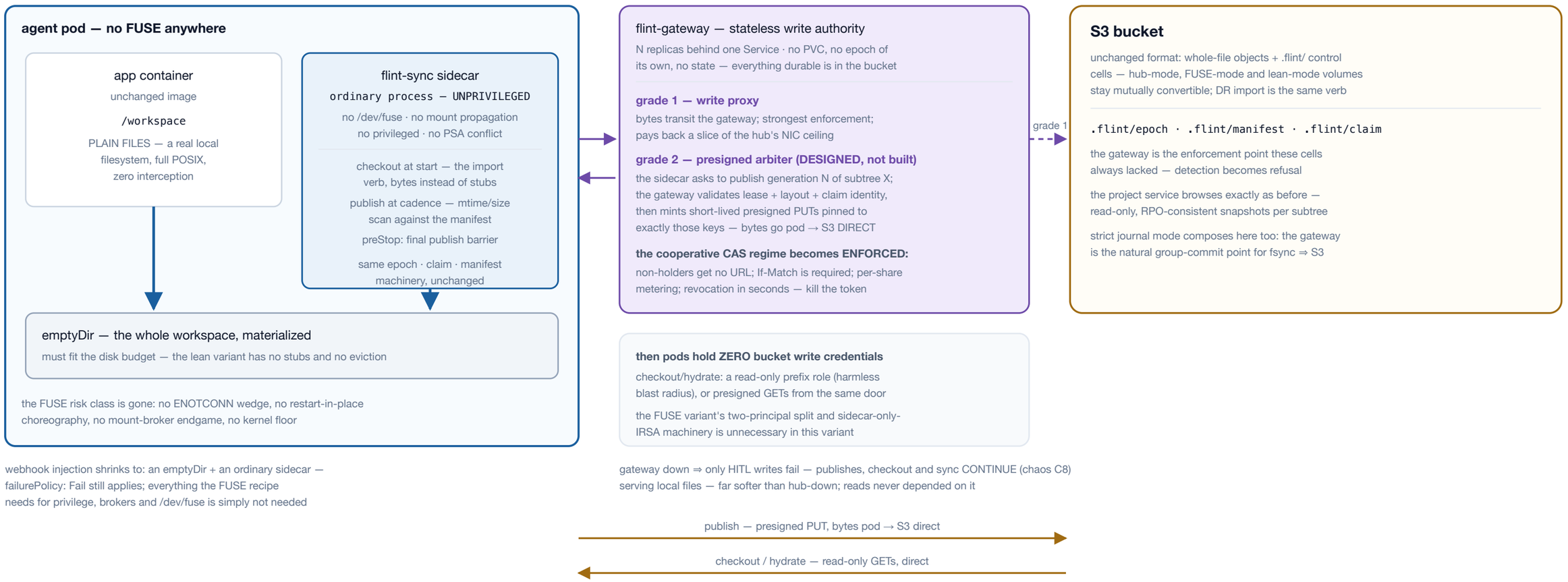
# Flint-lean — plain files, one bucket



The third front end over the flint bucket format, split out of the flint-fuse architecture doc: if the workspace fits on disk, no interception is needed at all. Materialize plain files at pod start, publish changed files at cadence with an unprivileged sidecar, and put write authority in a stateless gateway. Most of FUSE's value at a fraction of its risk surface — same bucket format, same engine.




### what the lean variant gives up — FUSE is the escalation, not the default

**lazy hydration**
cold start downloads the whole workspace; fine at GiB scale — a partial-checkout path list covers bigger trees

**interception-grade capture**
scan-based publish re-uploads touched files whole — the engine's whole-dirty fallback was already the correctness backstop

**eviction / space management**
no stubs: the workspace must fit the emptyDir budget; TiB-scale or lazy volumes are FUSE's territory (its own doc)

**This is a third front end, not a FUSE configuration** — the app touches plain files on a real local filesystem, and the sidecar is an ordinary unprivileged program running the same engine verbs: `import` at start (bytes instead of stubs), the flush barrier's whole-file lane at cadence (driven by an mtime/size scan against the manifest instead of capture interception), the same epoch, claim, and manifest machinery. For the agent-workspace shape — code, configs, artifacts, GiB-scale — it is most of FUSE's value with none of FUSE's hard parts: the entire privileged/mount-propagation/ENOTCONN/mount-broker risk section of the FUSE architecture stops applying, and the kernel floor drops to nothing. **Two corrections from the drills.** This page originally read “gateway down ⇒ publishes pause”; chaos leg C8 measured the opposite — barriers continue, because the shipped sidecar writes its manifest, window and epoch cells straight to the store. All four stated outage effects belong to the **proxy**; the gateway costs only the fourth. And **grade 2 is a design, not a build** — nothing in the tree mints a presigned URL today, and the shipped path gives the sidecar a scoped credential of its own (the agent container still holds none). **The gateway restores the security posture the review said direct mode gives up.** With grade 2, "the bucket trusts one principal" is true again — pods hold no write credentials, the cooperative CAS regime becomes enforced arbitration (a non-holder is refused a URL, not merely detected after the fact), revocation returns to seconds, and per-share metering exists again. Its failure mode is publish-pause with growing RPO while pods keep serving — the softest failure shape any component on these pages has. **One dialect everywhere:** this also argues for converging the hub's file API onto the S3 REST surface (the libflint plan's §8 sidecar adapter, built once, placed three ways — hub listener, localhost sidecar, gateway), so the live view and the published view speak the same protocol and the same tools work against both. **Recommendation:** lean is the direct-mode entry point; FUSE is the escalation for volumes too large or too lazy to materialize; the hub is unchanged. Same bucket, three front ends, one engine. Implementation plan: docs/plans/flint-lean-plan.md.

# One dialect — the S3 surface, three placements

The decision of record: every REST surface in the system speaks the S3 dialect — the hub's file API converges onto it, the gateway serves it, and direct-mode non-mount access (lean and FUSE alike) is plain S3 against the bucket. One library, three placements; the consistency contracts say which view a client is reading.

### s3_surface — built once

`SigV4 (header + presigned) · bearer`

`ListObjectsV2 · Get/Put/Head/Delete · Range`

`If-Match · If-None-Match:* · DeleteObjects`

`multipart: parts 1–10000 · ≥5 MiB · reorder`

consolidates libflint §8's localhost-gateway floor — one implementation instead of three

conformance kit runs against AWS and MinIO

the store's bucket-wide MinIO listing workaround regains its server-side Prefix here

### hub :8080 — the LIVE view

file API v2: strong, no RPO lag, atomic rename
dual-serves /files for one release, then deprecates
every data op counts as activity — polls do not

### flint-gateway — the fleet door

grade-1 S3 proxy with policy, plus the ONE non-S3 verb: the grade-2 publish grant (presigned mint)
bearer auth · seconds-scale revocation

### localhost S3 sidecar — later

apps that already speak S3, zero kernel path — libflint §8 adapter 3: the same library, placed a third way

### extensions the dialect adds — each one deliberate

| | |
|---|---|
| `RenameObject` | the Express One Zone verb, served on general buckets: ATOMIC on the hub's live view — something no real GP S3 offers |
| `x-amz-meta-flint-*` | posix stamps (mode/uid/gid/mtime) — the A12 convention |
| `activity classes` | data ops wake and keep a share awake; HEAD and list polls do not — a polling browser must not pin a share Ready forever |
| `publish grant` | POST intent → presigned set — grade 2's only custom verb |

### the multipart traps are requirements, not surprises

the ETag is md5-of-part-md5s + "-N", never a content MD5 — the s3proxy #338 class
CompleteMultipartUpload can return 200 WITH an error in the body — unhandled, SDK retries corrupt state
complete-by-concatenation costs 3× IO — reserve-and-copy is an explicit decision, since part sizes are unknown until receipt

### two views, one protocol — the consistency contracts tell them apart

#### the hub endpoint — the LIVE view

strong: close-to-open, locks behind it, atomic rename, no RPO lag — reads and writes land on the working set through the coherence authority

who: front doors, editors, anything touching a LIVE shared tree

#### the bucket endpoint — the PUBLISHED view

RPO-consistent snapshots per subtree; untorn whole-file generations; read-only against live subtrees (a console PUT has three bad fates — consistency doc)

who: project browsers, DR, analytics, direct-mode non-mount access

direct-mode REST access — lean AND FUSE variants alike — is the bucket endpoint: plain S3, no flint server anywhere in the read path.
the same rclone / boto3 / aws-cli invocation works against either endpoint; only the URL and the consistency contract change.

**Why S3 and not a nicer bespoke API:** the ecosystem is the feature — every language has a mature S3 client, presigned flows are understood by every CI system and browser uploader, and the tools people already point at buckets (rclone, boto3, cyberduck, Velero) work against a live flint share the day the dialect ships. The file API was already shaped on S3 semantics — whole-file GET/PUT with ETag and If-Match (the v1.30 conditional-writes work maps one-to-one onto S3's own conditional headers) — so this is a dialect change, not an architecture change. **The one thing the hub's live view offers that no real S3 can:** assembly under a temp name completed by an atomic RENAME under a deny-both OPEN — an atomic, exclusive multipart-complete, the differentiator the libflint plan already claims for the localhost gateway, inherited here. **Migration is dual-serve:** the hub answers both `/files` and the S3 dialect for one release; the front door moves; `/files` deprecates. Bearer auth stays alongside SigV4 so existing gateway tokens keep working, and revocation keeps its seconds-scale semantics. The activity classification is the one place the dialect must be flint-aware from day one — the idle ladder's lesson stands: a conditional GET answering 304 pins a share exactly as hard as a 200, so what counts as "activity" is declared per verb, not inferred.

# Boundary verbs — the agent says when



Pages 1–2 describe publishing **at cadence**: an mtime/size scan on a timer. That is still the default, and it is still the whole story for a workspace nobody is watching. What it could not express is a coherent point — "these files, together, now" — so an agent finishing a unit of work had no way to say so and no way to learn when its bytes were durable. The boundary verbs are that sentence, written as files in the workspace the agent already has. Nothing new is mounted, no port is opened, and a workspace that ignores them pays nothing: measured at **20 bucket requests per 22 s idle with the verbs off and 20 with them on**.

### the workspace — the whole interface

**agent writes**

`.flint/publish`
`.flint/sync`

a nonce in a file — no port, no client, no credentials

→

**sidecar answers**

`.flint/publish.ack`

status ok | partial | `refused-fenced`

an ok ack means the bytes are in S3

**news without a request**

`.flint/remote.seq`

observed_seq > integrated_seq means "somebody else published; consider a sync". Fed by the barrier's own HEAD, so N agents cost ZERO extra bucket requests, and a heartbeat field separates "no news" from "sidecar dead".

### gated mode — durability split from visibility

**the upload lane**

runs continuously; every changed file becomes a NEW OBJECT VERSION

durable immediately — and invisible to readers

a crash here loses nothing

→

**the citation — ONE CAS**

installs the whole pending set at once, naming version ids

a reader sees the whole change or none of it — never a mixture

no copy phase, so no half-boundary

**when a citation fires**

the agent asks (a sentinel) · the tree goes quiet · a lag cap expires · a backlog cap trips · preStop drains

gated is REFUSED without a lag bound, so unbounded staleness is impossible by construction, not by convention

### the operator — one CRD, one webhook, no new data path

A FlintLeanWorkspace names the bucket, the prefix and the knobs; the mutating webhook injects the sidecar as a NATIVE sidecar (initContainer, restartPolicy Always) so the workspace is materialized before the agent's first line runs, and an opted-in pod whose workspace is missing or refused does not schedule at all. The webhook serves in-process and provisions its own TLS. Observed state rides the lease heartbeat that was already happening: cited seq, staged-uncited count, the mode actually running.

### what is NOT here, and deliberately

· No new component in the data path. The verbs are files; the doors (a pod-local UNIX socket, a gateway endpoint) are opt-in sugar over the same consume path.

· No fsync interception. A boundary is declared, never inferred — that is the whole reason this is not FUSE.

· No enforced data-plane fencing. A deposed sidecar is refused at the control plane and its writes land as UNCITED versions: they destroy nothing and no coherent reader resolves them, but a raw-key reader can still see them. Closing that needs per-request epoch validation at a proxy, which is a component this front end exists to avoid — so it is a chosen ceiling, not an unfinished task.

---

**The verbs are files because the workspace is the interface.** An agent that can write a file can declare a boundary — no client library, no credential, no network path, and the same mechanism serves a shell script and a language runtime. The ack is the other half: `ok` means the boundary is installed in the bucket, `partial` means it installed without something the agent declared, and `refused-fenced` means this sidecar lost the lease and is saying so rather than leaving the agent waiting — which it used to do, forever. **Gated mode answers a different question from cadence:** cadence asks how stale a reader may be, gated asks whether a reader may see half a logical change, and answers no by making durability and visibility separate events. The cost is stated, not hidden — uncited versions exist in the bucket, a raw-key reader can see them, and the retention backstop must outlast the crash window or it will reap a version the manifest still cites. **Evidence, not assertion:** 27 bucket legs against a real MinIO, 14 operator legs on kind, a 61-run TLC gate, and the published chart installed on a three-node AWS cluster publishing to real S3 over TLS — logs committed under `lean/e2e/results/`.

# Does it fit? — the workflow lean is for



Lean is not a general filesystem and does not try to be. It is for **one agent, one workspace, one writer, a workspace that fits on disk** — the harness shape: clone-ish tree of code and artifacts, an agent that reads and writes it locally at full speed, and a durable point somebody else can read when the work is done. Everything below is the boundary of that claim, in measured numbers, so the answer to “will this work for us” is arithmetic rather than opinion.

### three questions, in order

**1 · does the workspace fit the disk?**
the whole tree is materialized — no stubs, no eviction.
No ⇒ FUSE (its own doc). This is the hard gate.

**2 · is one writer per subtree enough?**
a lease admits exactly one sidecar; a second is refused, not merged. No ⇒ the hub, for live shared POSIX.

**3 · is snapshot freshness acceptable?**
readers see the last published boundary, not your last write. No ⇒ the hub. Yes ⇒ lean, and read on.

### the loop — one agent, start to durable

**pod starts** → **agent works** → **agent declares** → **sidecar answers**

- **pod starts** — webhook injects the sidecar; checkout runs BEFORE the agent's first line
- **agent works** — plain local files, full POSIX, zero interception — this is the part that is just fast
- **agent declares** — `echo > .flint/publish` — one file write — the whole API surface
- **sidecar answers** — `.flint/publish.ack` — ok ⇒ the bytes are in S3 and a sibling can read them

**what it costs — MEASURED, loopback floors (0b); a latency-bound proxy moves these**

| | |
|---|---|
| `bytes` | checkout 3.3 s/GiB · publish 8.0 s/GiB |
| `100k files` | checkout 49.5 s · first publish 65 s (was 117 s before bounded fan-out) · idle tick 1.85 s |
| `1M files` | checkout 7 m 05 s · manifest 264 MiB at ~277 B/entry, exactly linear — the manifest is the wall |
| `v1 cap` | ~250k files, and it falls out of the manifest size, not the byte count |

idle costs ~5 bucket requests per tick, and the verbs add ZERO when unused — measured 20 vs 20 over 22 s

### worked example — a coding-agent fleet

A 2 GiB repo, 40k files, 30 agent pods, each on its own prefix. Checkout ≈ 7 s of bytes plus ~20 s of file count, so a pod is working inside half a minute. Each agent publishes when it finishes a task rather than on a timer, so a reviewer reads whole units of work, never a half-written tree. Idle pods cost about five requests a tick each. Nothing is shared live: if two agents must edit one tree simultaneously, that is the hub's job, not this one — and the lease will tell you so by refusing the second writer rather than merging them badly.

### when to walk away from lean

· The tree does not fit the pod's disk, or is TiB-scale, or you need lazy access to a small slice of a huge dataset — FUSE.
· Two writers must see each other's bytes live, or you need byte-range locks, O_EXCL or shared sqlite/git ACROSS pods — the hub.
· A reader cannot tolerate seeing the last boundary instead of the last write — the hub. Gated mode narrows WHAT a reader sees, never HOW STALE it is.

**The fit question is one question, and it is about the disk.** Everything else follows: because the whole tree is local, the app gets real POSIX at local speed and the FUSE risk class disappears; because it is local, it must fit; and because publishing is a boundary rather than a write-through, a reader sees coherent points instead of a live tree. **The file count binds before the byte count.** At ~277 B per manifest entry the document is exactly linear, so a million small files costs a 264 MiB manifest while a million bytes costs nothing — which is why the v1 cap is expressed in files (~250k) and why the levers that mattered were fan-out and skipping the manifest parse on an idle tick, not raw throughput. **The verbs change the shape of the workflow, not its cost.** Cadence alone forces a choice between publishing too often (wasted work) and too rarely (a reviewer reads a half-finished tree); a declared boundary removes the choice, and costs nothing when unused. **Say no early.** The three questions above are ordered so the cheapest disqualifier comes first, and a workload that fails any of them is better served by another front end over the same bucket format — all three are mutually convertible, so choosing lean today does not strand the data if the answer changes.